

By: Batoul Tamer 231000822  
Under supervision of: Dr. Muhammed Elsayeh

|            |                                                         |
|------------|---------------------------------------------------------|
| Dataset    | UCI Diabetes 130-US Hospitals (1999–2008)               |
| Records    | 101,766 patient encounters across 130 US hospitals      |
| Features   | 50 original features → 20 selected features             |
| Task       | Binary Classification — Readmitted within 30 days       |
| Best Model | XGBoost (Tuned) — GridSearchCV optimized                |
| Tools      | Python, Pandas, Scikit-learn, XGBoost, SMOTE, Streamlit |

0.8429

F1 Score

0.9944

ROC-AUC

88.9%

Recall

80.2%

Precision

## 1. Introduction & Problem Statement

### 1.1 Background

Hospital readmission — defined as a patient returning to hospital within 30 days of discharge — represents one of the most significant challenges in modern healthcare. In the United States alone, preventable readmissions cost the healthcare system an estimated **\$26 billion annually**. For diabetic patients specifically, the readmission rate within 30 days stands at approximately **11%**, making it a critical area for intervention and prediction.

The 30-day readmission window is recognized as the standard benchmark by hospitals, health insurers, and government bodies such as the Centers for Medicare and Medicaid Services (CMS). Readmission within this window is widely considered a preventable failure in care quality, and hospitals face financial penalties when readmission rates are deemed too high.

### 1.2 Problem Statement

Currently, most hospitals lack a systematic data-driven tool to identify which patients are at high risk of readmission before they are discharged. Clinical decisions are often made based on intuition and experience rather than predictive analytics. This creates a missed opportunity — if a high-risk patient is identified before discharge, targeted interventions (follow-up appointments, medication reviews, home care arrangements) can significantly reduce the likelihood of readmission.

### 1.3 Objective

This project aims to build a complete end-to-end machine learning pipeline that:

- Predicts whether a diabetic patient will be readmitted within 30 days of hospital discharge
- Uses only patient information available at the time of discharge — no future data leakage
- Achieves clinically meaningful performance with high Recall (catching most true readmissions) •  
Is deployed as an interactive web application accessible to healthcare providers

### 1.4 Dataset

The dataset used is the **UCI Diabetes 130-US Hospitals dataset (1999–2008)**, originally published by Strack et al. (2014) and available on Kaggle. It contains 101,766 patient encounters from 130 US hospitals and integrated delivery networks, with 50 features spanning patient demographics, clinical measurements, diagnostic codes, medication records, and utilization history.

## 2. Data Understanding & Cleaning

### 2.1 Dataset Overview

The dataset comprises 50 columns organized into 6 main feature categories:

| Category            | Features                                                              | Description                                    |
|---------------------|-----------------------------------------------------------------------|------------------------------------------------|
| Demographics        | race, gender, age                                                     | Patient background — age stored as string bins |
| Admission Info      | admission_type, discharge_disposition, admission_source               | How/where admitted and discharged              |
| Hospital Stay       | time_in_hospital, num_lab_procedures, num_procedures, num_medications | Clinical activity during the visit             |
| Diagnoses           | diag_1, diag_2, diag_3, number_diagnoses                              | ICD-9 coded diagnoses — up to 3 per visit      |
| Medications (24)    | metformin, insulin, glipizide, glyburide + 20 more                    | Diabetes medication adjustments                |
| Utilization History | number_outpatient, number_emergency, number_inpatient                 | Prior healthcare usage in the past year        |

## 2.2 Data Quality Issues Found

A critical first step was discovering that the dataset, despite appearing clean on the surface, contained several serious quality problems. A standard `df.isnull()` check returned zero missing values — which was completely misleading. The actual issues were:

| Issue                  | Severity | Affected    | Description                                                      |
|------------------------|----------|-------------|------------------------------------------------------------------|
| Hidden '?' strings     | CRITICAL | 97,000+     | Missing values stored as '?' — invisible to standard null checks |
| weight: 97% missing    | HIGH     | 98,569 rows | Nearly entire column missing — no imputation possible            |
| medical_specialty: 49% | HIGH     | 49,949 rows | Too sparse to use reliably as a feature                          |
| Duplicate patients     | HIGH     | 30,248      | Same patient with multiple visits — causes data leakage          |
| Age as string bins     | MEDIUM   | All rows    | '[30-40)' strings — must convert to numeric midpoints            |



|                           |        |            |                                                           |
|---------------------------|--------|------------|-----------------------------------------------------------|
| Raw ICD-9 codes           | MEDIUM | 3 columns  | Codes like '250.83' — meaningless without grouping        |
| Class imbalance           | MEDIUM | 11% target | Only 11% of patients readmitted — accuracy metric useless |
| Zero-variance medications | LOW    | 14 columns | 95-100% 'No' — add noise without predictive value         |

### 2.3 Cleaning Actions Taken

Each issue was addressed with a justified, documented action:

|                                                                                                      |                                                                              |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <div>1. Replace '?' with NaN</div> <div>df.replace('?', np.nan)</div>                                | Makes all missing values properly detectable for subsequent handling         |
| <div>2. Drop high-missing columns</div> <div>weight, medical_specialty, payer_code dropped</div>     | Columns with >40% missing — no imputation strategy is reliable at this level |
| <div>3. Engineer history features</div> <div>Before deduplication — 4 features from all visits</div> | Preserves patient history information before duplicate rows are removed      |
| <div>4. Deduplicate patients</div> <div>Sort by encounter_id, keep first visit per patient_nbr</div> | Reduces 101,766 → 71,518 rows, eliminates data leakage from future visits    |
| <div>5. Fill remaining NaNs</div> <div>AlCresult, max_glu_serum → 'None'; race → 'Unknown'</div>     | NaN means test was not performed — 'None' is the clinically accurate label   |
| <div>6. Fix age column</div> <div>[30-40) → 35, [50-60) → 55, etc.</div>                             | Converts string bins to numeric midpoints for model compatibility            |

|                                                                                           |                                                                       |
|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
|                                                                                           | Domain knowledge grouping makes diagnosis codes meaningful and usable |
| <b>7. Group ICD-9 codes</b><br><br>8 categories: Circulatory, Diabetes, Respiratory...    |                                                                       |
| <b>8. Create binary target</b><br><br>readmitted_binary = 1 if readmitted <30 days else 0 | Converts 3-class problem to clinically meaningful binary prediction   |
| <b>9. Drop zero-variance medications</b><br><br>14 columns where >95% values are 'No'     | Low-variance features add noise without predictive signal             |
| <b>10. Handle class imbalance</b><br><br>SMOTE applied on training set only               | Balances the 11%/89% split — critically NOT applied to the test set   |

2.4 Before vs After Cleaning

| Metric             | Before Cleaning | After Cleaning | Change                               |
|--------------------|-----------------|----------------|--------------------------------------|
| Total rows         | 101,766         | 71,518         | -30,248 (duplicate patients removed) |
| Total columns      | 50              | 33             | -17 (dropped low-value columns)      |
| Missing values     | 97,000+ hidden  | 0              | All resolved                         |
| Duplicate patients | 30,248          | 0              | Deduplicated                         |
| Target balance     | 11% positive    | 50% (train)    | SMOTE applied on train only          |

3. Exploratory Data Analysis

3.1 Analysis Approach

Exploratory Data Analysis (EDA) was conducted to understand the distribution of variables, identify patterns, and generate hypotheses about which features are most likely to predict readmission. Both **univariate analysis** (examining each variable individually) and **bivariate analysis** (examining relationships between variables and the target) were performed.

| Plot# | Plot Type           | Variable(s)                    | Analysis Type | Key Finding                                 |
|-------|---------------------|--------------------------------|---------------|---------------------------------------------|
| 1     | Bar Chart           | readmitted_binary              | Univariate    | 11% positive class — severe imbalance       |
| 2     | Histogram           | age                            | Univariate    | Most patients aged 60-80                    |
| 3     | Box Plot            | time_in_hospital               | Univariate    | Median 4 days, right-skewed                 |
| 4     | Horizontal Bar      | diag_1 (categories)            | Univariate    | Circulatory is #1 diagnosis                 |
| 5     | Box Plot            | age vs readmitted              | Bivariate     | Readmitted patients 5yrs older on avg       |
| 6     | Violin Plot         | num_medications vs readmitted  | Bivariate     | Readmitted patients take more medications   |
| 7     | Bar Chart           | race vs readmission rate       | Bivariate     | Variation across racial groups              |
| 8     | Correlation Heatmap | All numeric features           | Multivariate  | Prior visits most correlated with target    |
| 9     | Count Plot          | number_inpatient vs readmitted | Bivariate     | Strong positive trend with inpatient visits |

3.2 Key Findings from EDA

• Class Imbalance

Only 11,357 out of 101,766 patients (11%) were readmitted within 30 days. This makes accuracy a misleading metric — a model predicting 'no readmission' for all patients achieves 89% accuracy while catching zero actual readmissions. This finding drove our decision to use Precision, Recall, and F1-Score as evaluation metrics, and to apply SMOTE for class balancing.

• Age as a Risk Factor

Readmitted patients are on average 5 years older than non-readmitted patients. The 70–80 age group shows the highest readmission rates, likely due to more comorbidities, slower recovery, and greater care complexity in elderly patients.



• **Prior Inpatient Visits**

Patients with 2 or more prior inpatient visits in the past year are approximately 3 times more likely to be readmitted. This finding directly motivated the creation of the `total_encounters`, `max_inpatient_history`, and `high_utilizer` engineered features.

• **Medication Complexity**

Readmitted patients take an average of 16 medications compared to 13 for non-readmitted patients. Higher medication counts indicate more complex or unstable medical conditions and serve as a proxy for disease severity.

• **Emergency Visit History**

Patients with any prior emergency visit have more than double the readmission odds compared to those with none. This strongly supported the creation of the `has_emergency_history` binary feature.

• **Primary Diagnosis Distribution**

Circulatory disease is the most common primary diagnosis category, followed by Other and Respiratory conditions. Notably, Diabetes itself (ICD-9 code 250.xx) appears less frequently as the primary code — most diabetic patients are hospitalized for complications rather than diabetes directly.

4. Feature Engineering & Selection

4.1 Encoding Categorical Variables

Before engineering new features, categorical columns were properly encoded. A critical distinction was made between **nominal** and **ordinal** categories:

| Encoding Type          | Columns                                                        | Rationale                                                                        |
|------------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------|
| One-Hot Encoding       | race, gender, diag_1, diag_2, diag_3, max_glu_serum, A1Cresult | Nominal categories — no natural order. Label encoding would imply false ranking. |
| Label/Ordinal Encoding | metformin, insulin, glipizide, glyburide + other medications   | Ordinal: No < Steady < Down/Up — natural direction of medication change exists.  |
| Binary Encoding        | change, diabetesMed                                            | Yes/No binary variables — directly encoded as 1/0.                               |

4.2 New Features Created

Eight new features were engineered, each designed with a specific clinical rationale. The first four were created from all patient encounters **before deduplication** to capture history without causing data leakage:

| Feature Name             | Type        | Formula                                     | Clinical Rationale                                                           |
|--------------------------|-------------|---------------------------------------------|------------------------------------------------------------------------------|
| total_encounters         | History     | count(all visits per patient)               | Frequent admissions = chronic instability — strongest predictor found        |
| ever_readmitted          | History     | 1 if patient ever had <30d readmission      | Past readmission is the best clinical predictor of future readmission        |
| max_inpatient_history    | History     | max(number_inpatient) across all visits     | Captures worst-case inpatient burden across the patient's full history       |
| mean_medications_history | History     | mean(num_medications) across all visits     | Chronic high medication use indicates complex, unstable conditions           |
| total_services           | Complexity  | lab_procedures + procedures + medications   | Single complexity score for the visit — high values = more serious admission |
| has_emergency_history    | Risk Flag   | 1 if number_emergency > 0<br>else 0         | Any prior emergency visit doubles readmission odds (from EDA)                |
| num_meds_changed         | Instability | count of medications with Up or Down status | Multiple changes = medically unstable patient at discharge                   |
| high_utilizer            | Risk Flag   | 1 if inpatient ≥ 2 OR emergency ≥ 2         | Combined flag — patients who repeatedly use acute care services              |

### 4.3 Feature Selection — Two Methods Applied

Starting with 40+ features after encoding, two sequential selection methods were applied to identify the 20 most predictive features:

#### Method 1 — Filter Method (Correlation Threshold)

Pearson correlation between every feature and the target variable was calculated. Features with absolute correlation below 0.01 were removed. This method is fast, model-independent, and removes obviously uninformative features before expensive model fitting. Approximately 12 near-zero correlation features were eliminated.

#### Method 2 — Embedded Method (Random Forest Feature Importance)

A Random Forest classifier was trained on the filtered feature set, and built-in feature\_importances\_ scores were extracted. These measure how much each feature reduces Gini impurity across all trees.



The top 20 features by importance score were retained for final modeling.

Top 10 features by Random Forest importance score:

| Rank | Feature                  | Importance Score | Category                |
|------|--------------------------|------------------|-------------------------|
| 1    | ever_readmitted          | 0.089            | Engineered — History    |
| 2    | total_encounters         | 0.071            | Engineered — History    |
| 3    | number_inpatient         | 0.068            | Original                |
| 4    | max_inpatient_history    | 0.062            | Engineered — History    |
| 5    | num_lab_procedures       | 0.058            | Original                |
| 6    | num_medications          | 0.055            | Original                |
| 7    | total_services           | 0.051            | Engineered — Complexity |
| 8    | age                      | 0.048            | Original                |
| 9    | number_diagnoses         | 0.044            | Original                |
| 10   | mean_medications_history | 0.041            | Engineered — History    |

Notably, 4 of the top 10 features are engineered features — confirming that the feature step added genuine predictive value beyond the original columns.

## 5. Model Training & Comparison

### 5.1 Experimental Setup

The following setup was used consistently across all three models:

| Parameter        | Value                  | Rationale                                                                           |
|------------------|------------------------|-------------------------------------------------------------------------------------|
| Train/Test Split | 80% / 20% (stratified) | Stratify=y preserves the 11% positive class ratio in both splits                    |
| Class Balancing  | SMOTE on train only    | Synthetic oversampling of minority class — test set left untouched                  |
| Feature Scaling  | StandardScaler         | Normalizes features — critical for Logistic Regression, applied to all for fairness |
| Random State     | 42 (all models)        | Ensures reproducibility across all experiments                                      |
| Evaluation Set   | Held-out test set only | Test set never seen during training or tuning                                       |

### 5.2 Algorithms Compared

#### 5.2.1 Logistic Regression (3rd Place)

A linear classifier that models the log-odds of readmission as a weighted sum of features. Used as a baseline to establish minimum acceptable performance and provide interpretability. Assumes a linear decision boundary — appropriate as a starting point for comparison.

| Strengths                                                                                                                                                                                              | Limitations                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>✓ Very fast training — seconds on 57K rows</li><li>✓ Highly interpretable — coefficient = feature weight</li><li>✓ Well-calibrated probability outputs</li></ul> | <ul style="list-style-type: none"><li>X Cannot capture non-linear feature interactions</li><li>X Assumes feature independence</li><li>X Lower ceiling performance on complex clinical data</li></ul> |



5.2.2 Random Forest (2nd Place)

An ensemble of 100 decision trees, each trained on a random subset of data and features (bagging). Final prediction is the majority vote across all trees. The randomization makes it resistant to overfitting compared to a single deep tree.

| Strengths                                                                                                                                                          | Limitations                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>✓ Handles non-linear relationships naturally</div><div>✓ Resistant to overfitting via bagging</div><div>✓ Built-in feature importance scores</div></div> | <div><div>X Slower to train than Logistic Regression</div><div>X Less interpretable — 100 trees are opaque</div><div>X Memory intensive for large datasets</div></div> |

5.2.3 XGBoost (Winner)

Gradient Boosted Trees — an ensemble where trees are built sequentially, each correcting the errors of the previous one using gradient descent in function space. XGBoost adds regularization (L1/L2) to control overfitting, making it more robust than basic boosting.

| Strengths                                                                                                                                                               | Limitations                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>✓ Best overall performance on this dataset</div><div>✓ Handles missing values natively</div><div>✓ Highly tunable — responds well to GridSearchCV</div></div> | <div><div>X More hyperparameters to tune</div><div>X Can overfit without regularization</div><div>X Less interpretable than Logistic Regression</div></div> |

5.3 Model Comparison Results

| Model               | Precision | Recall | F1 Score | ROC-AUC | Rank                                  |
|---------------------|-----------|--------|----------|---------|---------------------------------------|
| Logistic Regression | 0.7393    | 0.9754 | 0.8411   | 0.9902  | 3 <sup>rd</sup>                       |
| Random Forest       | 0.7683    | 0.9404 | 0.8457   | 0.9938  | 2 <sup>nd</sup>                       |
| XGBoost (default)   | 0.7794    | 0.9118 | 0.8404   | 0.9946  | 1 <sup>st</sup> — selected for tuning |

XGBoost was selected for tuning based on its highest Precision (0.7794) and ROC-AUC (0.9946), indicating the best discriminative ability. All three models substantially exceed the required 0.3 minimum threshold for both Precision and Recall.



## 6. Hyperparameter Tunning & Evaluation

### 6.1 What is Hyperparameter Tuning?

Machine learning models have two types of parameters. **Parameters** are learned from data during training (e.g., tree split thresholds). **Hyperparameters** are set before training and control how the model learns (e.g., number of trees, learning rate). Default hyperparameter values are generic — they are reasonable starting points but are not optimized for any specific dataset. Tuning systematically searches for the combination of hyperparameter values that maximizes model performance on our specific data.

### 6.2 GridSearchCV with 5-Fold Cross-Validation

GridSearchCV was used to evaluate every combination of hyperparameters in a predefined grid. Each combination was evaluated using 5-fold cross-validation on the training set — the training data is split into 5 equal folds, the model trains on 4 and validates on 1, rotating 5 times. This produces a robust performance estimate while preventing overfitting to a single split. F1-Score was used as the optimization objective given the class imbalance.

| Hyperparameter | Values Tested    | Description                                                           |
|----------------|------------------|-----------------------------------------------------------------------|
| n_estimators   | [100, 200, 300]  | Number of boosting trees — more trees = more learning capacity        |
| max_depth      | [3, 5, 7]        | Maximum depth per tree — controls complexity and overfitting          |
| learning_rate  | [0.01, 0.1, 0.2] | Step size — smaller = more conservative learning, lower overfit       |
| subsample      | [0.8, 1.0]       | Fraction of training data per tree — adds randomness, reduces overfit |

Total combinations:  $3 \times 3 \times 3 \times 2 = 54$  combinations  $\times$  5 folds = 270 training runs.

Best parameters found: **n\_estimators=200, max\_depth=5, learning\_rate=0.1, subsample=0.8**

### 6.3 Tuning Results — Before vs After

| Metric | XGBoost Default | XGBoost Tuned | Change | Interpretation |
|--------|-----------------|---------------|--------|----------------|
|--------|-----------------|---------------|--------|----------------|

|           |        |        |          |                                                          |
|-----------|--------|--------|----------|----------------------------------------------------------|
|           |        |        |          |                                                          |
| Precision | 0.7794 | 0.8016 | +0.022 ✓ | Fewer false alarms — more reliable predictions           |
| Recall    | 0.9118 | 0.8888 | -0.023   | Slightly fewer readmissions caught — acceptable tradeoff |
| F1 Score  | 0.8404 | 0.8429 | +0.003 ✓ | Better overall balance between precision and recall      |
| ROC-AUC   | 0.9946 | 0.9944 | -0.0002  | Near-identical discrimination ability                    |

6.4 Evaluation Metrics Explained

Given the class imbalance (89% negative, 11% positive), standard accuracy is not a meaningful metric. The following metrics were used instead:

• **Precision =  $TP / (TP + FP) = 0.8016$**

When the model predicts readmission, it is correct 80% of the time. Low precision means too many false alarms — healthy patients incorrectly flagged, wasting hospital resources on unnecessary interventions.

• **Recall (Sensitivity) =  $TP / (TP + FN) = 0.8888$**

Of all patients who were truly readmitted, the model caught 89% of them. In healthcare, recall is the most critical metric — a False Negative (missed readmission) means a high-risk patient is discharged without intervention, which is the most dangerous outcome.

• **F1 Score =  $2 \times (P \times R) / (P + R) = 0.8429$**

The harmonic mean of Precision and Recall — provides a single balanced metric that accounts for both. F1 is the standard metric for imbalanced classification problems.

• **ROC-AUC = 0.9944**

The probability that the model assigns a higher risk score to a truly readmitted patient than to a non-readmitted one. A score of 0.99 indicates near-perfect discrimination ability — the model almost always correctly ranks patients by their true risk level.

6.5 Validation Strategy

Validation ensures model performance estimates reflect real-world generalization. Two strategies were employed: (1) An 80/20 stratified train-test split where the test set was completely withheld during training and tuning — all reported metrics are computed on this unseen test set. (2) 5-fold cross-



validation within GridSearchCV to select hyperparameters robustly — preventing the tuning process itself from overfitting to a single data partition.

7. Deployment – Live Web Application

7.1 Application Overview

The tuned XGBoost model was deployed as an interactive web application using **Streamlit**, an open-source Python framework for building data applications. The application allows healthcare providers to enter patient information and receive an instant readmission risk prediction, making the model accessible without any programming knowledge.

7.2 How the Application Works

| Step | Action                     | Technical Detail                                                                                |
|------|----------------------------|-------------------------------------------------------------------------------------------------|
| 1    | Doctor enters patient data | Interactive sliders and dropdowns for age, medications, prior visits, diagnosis, discharge info |
| 2    | Feature engineering        | App recreates all 8 engineered features in real time using the same formulas from training      |
| 3    | Preprocessing              | Input data is scaled using the saved StandardScaler (scaler.pkl) from training                  |
| 4    | Model prediction           | Tuned XGBoost (model_xgb_tuned.pkl) returns probability and binary classification               |
| 5    | Results displayed          | HIGH RISK or LOW RISK classification with probability score and risk factors summary            |

7.3 Technical Stack

| Component         | Technology     | Purpose                                       |
|-------------------|----------------|-----------------------------------------------|
| Frontend          | Streamlit      | Interactive UI — sliders, dropdowns, buttons  |
| ML Model          | XGBoost        | Core prediction engine                        |
| Preprocessing     | Scikit-learn   | StandardScaler for feature normalization      |
| Data Handling     | Pandas / NumPy | Input data processing and feature engineering |
| Model Persistence | Joblib         | Saving/loading model and scaler as .pkl files |
| Version Control   | GitHub         | Code repository and project documentation     |



|         |                 |                                    |
|---------|-----------------|------------------------------------|
| Hosting | Streamlit Cloud | Free cloud deployment — public URL |
|---------|-----------------|------------------------------------|

7.4 Critical Implementation Note

A key challenge in deployment was ensuring that the input features passed to the model exactly match the columns used during training — both in name and order. The model was trained on 20 specific features in a specific column order. The app explicitly constructs a DataFrame with the exact same column names and order before calling `scaler.transform()` and `model.predict()`. Any mismatch causes a `ValueError` and must be resolved by printing `X_final.columns.tolist()` from the training notebook and matching it exactly in `app.py`.

## 8. Results Summary & Conclusion

### 8.1 Final Model Performance

|          |         |        |           |
|----------|---------|--------|-----------|
| 0.8429   | 0.9944  | 88.9%  | 80.2%     |
| F1 Score | ROC-AUC | Recall | Precision |

| Model               | Precision | Recall | F1     | AUC    | Status      |
|---------------------|-----------|--------|--------|--------|-------------|
| Logistic Regression | 0.7393    | 0.9754 | 0.8411 | 0.9902 | Baseline    |
| Random Forest       | 0.7683    | 0.9404 | 0.8457 | 0.9938 | Comparison  |
| XGBoost (default)   | 0.7794    | 0.9118 | 0.8404 | 0.9946 | Selected    |
| XGBoost (tuned)     | 0.8016    | 0.8888 | 0.8429 | 0.9944 | FINAL MODEL |

### 8.2 Project Contributions

This project made the following contributions beyond simply running a model:

• **Real Data Quality Investigation**

Discovered and documented 8 distinct data quality issues including hidden '?' nulls that a standard null check completely misses — demonstrating the importance of thorough data profiling before any modeling.

• **Clinically-Motivated Feature Engineering**

Created 8 new features grounded in clinical reasoning — not arbitrary transformations. 4 of the top 10 most important features were engineered features, validating their value.

• **Data Leakage Prevention**

Identified and resolved the duplicate patient encounter problem correctly — engineering history features from all visits before deduplicating, rather than naively dropping duplicates or including all visits and causing leakage.

### • Justified Decision-Making

Every cleaning action, modeling choice, and evaluation decision was documented with explicit rationale — not just what was done but why, enabling reproducibility and critique.

### • End-to-End Deployment

The project culminates in a live web application — demonstrating that machine learning value comes not from building models in isolation but from making them accessible and usable.

## 8.3 Limitations & Future Work

Several limitations of the current implementation should be acknowledged:

- The dataset is from 1999–2008 — clinical practices, medications, and patient demographics have changed significantly since then
- ICD-9 codes are no longer in use — modern systems use ICD-10, requiring updated mapping functions
- The model was trained on US hospital data and may not generalize to other healthcare systems
- Feature importance does not imply causality — clinical validation would be required before deployment in real hospitals
- The Streamlit app does not include patient history features (total\_encounters, ever\_readmitted) as inputs — these were proxied with simplified sliders

Potential directions for future improvement include:

- Incorporating ICD-10 diagnosis codes and more recent hospital data
- Exploring deep learning approaches (LSTM) for sequential patient visit modeling
- Adding SHAP (SHapley Additive exPlanations) values for individual prediction explainability
- Conducting prospective clinical validation with a healthcare partner
- Expanding the web app to accept full patient history across multiple visits

## 8.4 Conclusion

This project successfully demonstrated a complete machine learning pipeline for hospital readmission prediction — from raw, messy clinical data to a deployed, interactive web application. The final XGBoost model achieved an F1 Score of 0.8429 and ROC-AUC of 0.9944, substantially exceeding the required minimum thresholds and demonstrating clinically meaningful performance. The 89% Recall means the model correctly identifies the vast majority of patients who will be readmitted, giving healthcare providers actionable information to intervene before discharge.

More importantly, the project illustrated that data science is not merely about building models — it is about making careful, justified decisions at every step: understanding the problem domain, detecting



hidden data quality issues, engineering meaningful features, preventing data leakage, selecting appropriate metrics, and ultimately delivering something genuinely useful. Each of these steps

required deliberate thought, and this report has documented the reasoning behind every major decision made throughout the pipeline.

---

*Hospital Readmission Risk Prediction — Data Science Final Project | UCI Diabetes 130-US Hospitals Dataset  
(1999–2008)*